Week 5 – Design Studio: Answers

Design the simplified Amazon store front

> Read this after the studio. It isn't the only good answer; it's what a strong answer contains. The system is the ranking system from the lecture, with a customer and a page in place of a viewer and a session, and the differences are in the data: stock that changes by the minute, a purchase history that is both a signal and a filter, a grid instead of a list, and a slow truth, the return, that arrives weeks after the fast one.

The numbers, as given

The prompt supplies them; what a strong answer adds is the implication of each.

NUMBER	VALUE	IMPLICATION
requests at the peak	5,000 a second	the index lookup and the ranker batch must each be milliseconds; nothing per request may touch the catalog table
the grid	50 products in 300 ms	candidate generation returns a few thousand, the filters cut them, the ranker scores about a thousand in one batch, ordering is code
the catalog	8,000,000 products, 20,000 new or changed a day, stock by the minute	the product half is computed at the catalog change, not at the request; stock is a filter read at the request, not a feature baked in overnight
the customers	20% with no purchase history	the customer half cannot rely on purchases alone: views, the page, and the location have to carry the new customer
ground truth	click in seconds, purchase in the session, return in weeks	the fast label trains the ranker; the slow label corrects it and is a guardrail

1. Architecture

Before the request and at the request

When a product is added or changed. Its vector, from the product tower, into the vector database, which takes an insert: the vector joins its nearest cell's list. Its row in the products table (price, rating, category, sales over the last seven days, age in the catalog, whether it is bought again) is the catalog's own, kept by the catalog team. Twenty thousand times a day, nobody waiting.

Every minute. Stock by region, from the inventory system, into a small table the filters read. Not a feature of the ranker: a product that is in stock now and was not an hour ago should not be ranked by its hour-old self.

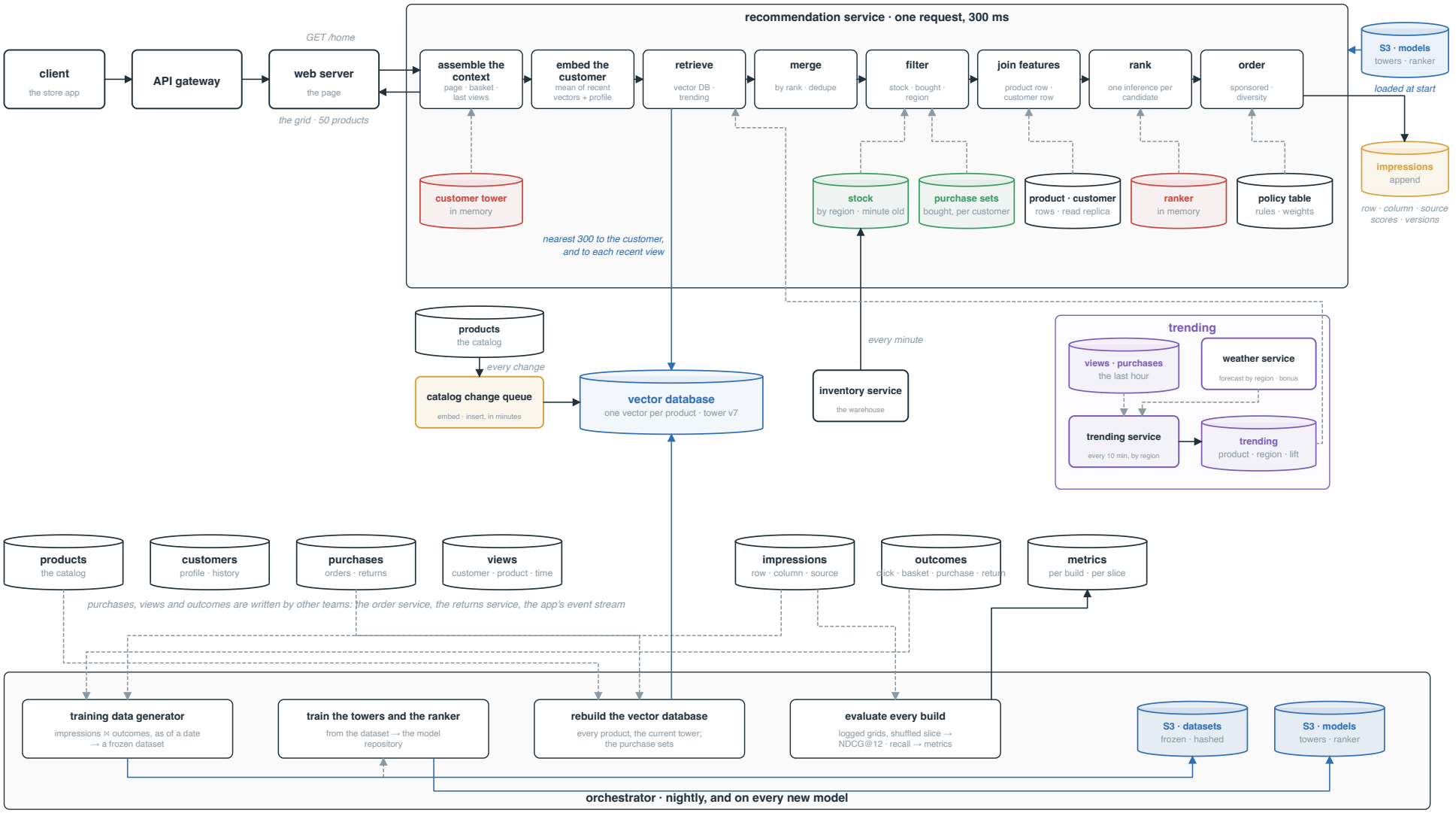
Every ten minutes. The trending list per location: what is being viewed and bought in this region right now, above its usual rate. The bonus is this job with one more input, the weather feed, so that a snowfall in Minneapolis lifts shovels before anyone has bought one.

At the request. Assemble the context (the page, the basket, the last few views); embed the customer; retrieve from several sources; merge; filter; add features; rank; order; log. About 50 ms of that is the ranker.

The seam. The vector database, the trending list, the stock table, and the products and customers tables, each with a version or an as-of time; the request path reads them and never writes them.

The architecture

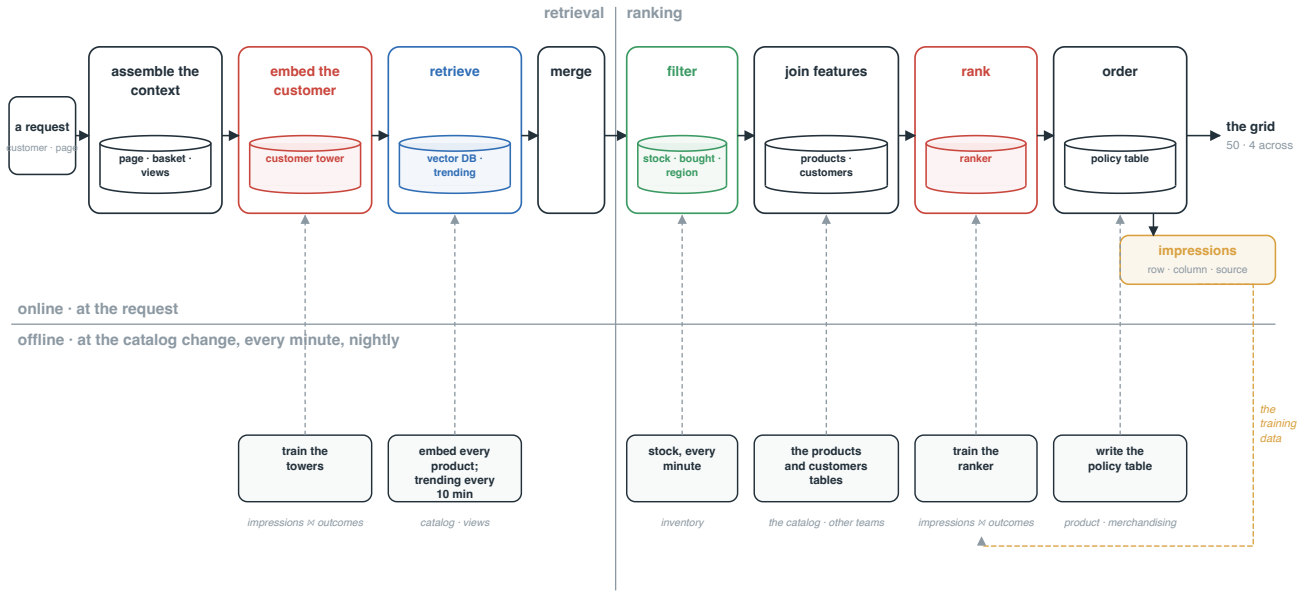
The components: what is one service, what is orchestrated, where the data flows. Solid arrows are the request path and writes; dotted arrows are reads; blue is a model or index artifact.



The store front: the services, the orchestrator, the stores, and the flow between them. Solid: the request path and writes. Dotted: reads. Blue: model and index artifacts.

Reading it: the recommendation service runs the whole request path in one process, with the customer tower, the ranker, the policy table, the stock table and the purchase sets in memory, the models loaded at start from the model repository. The retrieval stage leaves the process for the vector database and the trending list; the join stage reads the products and customers tables. The vector database holds one vector per product; every catalog change embeds the product and inserts it, so a new product is retrievable in minutes, and the nightly rebuild re-embeds everything with the current tower. Inventory pushes stock every minute; the trending job rebuilds its list every ten. The order stage appends every grid to the impression log with row, column and source. Purchases, returns, views and outcomes are written by other teams: the order system, the returns system and the app's event stream. The orchestrator runs four jobs: the training data generator that freezes a dataset, training of the towers and the ranker into the model repository, the rebuild of the vector database and the purchase sets, and the evaluation of every build on logged grids into the metrics table.

The same system on the two axes from the lecture, with the artifact each online stage reads:



The store front on two axes. Two candidate sources feed one merge; stock and the purchase set are filters read at the request; the grid is logged with row and column.

Stage by stage, with what the model changes:

STAGE	WHAT IT DOES	CODE OR MODEL
assemble the context	the page, the basket, the last twenty views, the customer's region and price band	a few reads from the customer's session and profile
embed the customer	the mean of the vectors of recent views and purchases, with the profile; the customer tower	a model, its version pinned with the product tower's
retrieve	nearest products to the customer vector · nearest to each of the last three viewed · trending in this region	the vector database for the first two; a table lookup for the third; each a few hundred with its reason attached
merge	by rank from every source, deduplicated, source attached	code
filter	out of stock in this region; already bought, unless repurchasable; not shipped here; outside the customer's price band by a wide margin; under a hold	code, from the minute-old stock table and the customer's purchase set; a rule no score may override
join features	the product's row from the products table; the customer's row from the customers table (tenure, categories, price band, purchases in 90 days); the pair (similarity, bought this brand, viewed this product); the context (page, hour, device)	two table reads, by id
rank	one inference per candidate, in one batch: p(click), p(add to basket), p(purchase), p(return)	the ranker
order	sponsored slots, at most four, marked, never the whole first row; one product per brand in the first row; no two from the same category in a row; the exploration slot; the grid written to the impression log with row and column	code, from the policy table, each rule with an owner

The vector database

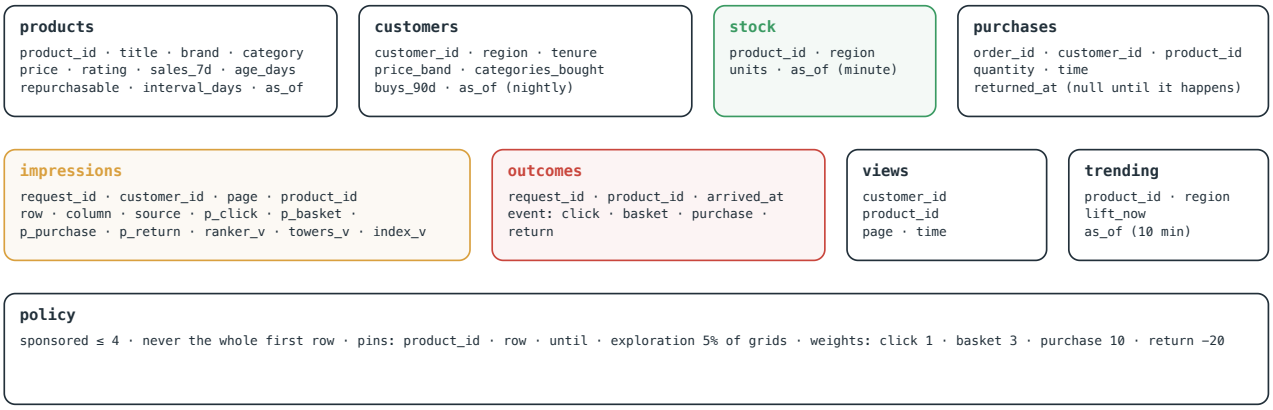
What is in it: one vector per product, 128 numbers, from the product tower, with the product id and the tower version; partitioned into cells around centroids. What writes to it: every catalog change, as an insert; the nightly rebuild, when the centroids are chosen again or the tower changes. What reads it: the retrieval stage, with a customer vector or a product vector, asking for the nearest few hundred at a chosen number of cells probed. What it adds beyond the bare index: inserts without a rebuild, a filter on region and category inside the search, a snapshot that survives a restart, and one version for the vectors, the centroids and the tower together.

The dials

How many cells the vector database probes; how many candidates each source returns; how many the ranker scores. Each trades recall for latency, and each is measured before it is set.

2. Data handling

The tables



The tables, and what one row holds. The impression row carries the grid position and the source; purchases carry the return when it arrives.

TABLE	ONE ROW	WRITTEN BY
products	one product: title, brand, category, price, rating, sales over seven days, age, repurchasable and its interval, as-of	the catalog team, on every change
customers	one customer: region, tenure, price band, categories bought, purchases in 90 days, as-of	the customer team, nightly
stock	one product in one region: units on hand, as-of	inventory, every minute
purchases	one line of one order: customer, product, quantity, time; and the return, if any, with its date	other teams: the order system, the returns system
views	one product viewed by one customer: time, page	other teams: the app's event stream
impressions	one product shown on one grid: request id, customer, page, product, row and column, source (customer, viewed, bought-together, trending, bestseller, sponsored, exploration), the ranker's probabilities, the versions of the ranker, the towers and the index	the order stage, the moment the grid renders
outcomes	one event: request id, product, click or basket or purchase or return, with `arrived_at`	the app's event stream, as each happens; the returns system, weeks later
trending	one product in one region: its rate now against its usual, as-of	the trending job, every ten minutes
policy	the sponsored rules, the pins, the exploration share, the combination weights	product and merchandising, with an owner per row

The training set, exactly

Impressions joined with outcomes on request id and product, as of a date. One row per product shown: the customer's row as served, the product's row as served, the pair features, the row and column, the source, the page, and the label. The label is the events with their weights from the policy table of that day: a click is worth little, a purchase a lot, a return negative. Kept out: anything the customer did on a later page, and any purchase of a product the grid did not show. Frozen with a hash and a number; the ranker carries the number.

The row and column are in the training row because the grid has its own effect: the first row is seen by everyone, the left column more than the right, and the bottom rows by whoever scrolled. A one-percent slice of grids shows the first three rows in shuffled order, which is what the correction is fitted on; without it the ranker learns that whatever was top-left was best.

A new product, and a customer with no history

A product added this morning has a title, a description, an image, a price and a category, and no clicks. The product tower embeds it from content, so the vector database can return it within minutes of the insert. Its products row has no sales: the catalog writes the category's average in its place and sets **age in catalog** to zero, which the ranker has a feature for. The exploration slot shows it to a few customers, and the impression rows it earns are how the next training run learns what it is.

A customer who has never bought anything has a region, a device, this session's views, and whatever is in the basket. The customer vector is the mean of the viewed products' vectors, so one view is enough to retrieve from; before any view, the trending list for the region and the bestsellers carry the grid. The history features take the defaults for a new customer, and the ranker has a feature for that too. The measurement section has a slice for exactly these customers.

The already-bought filter

The purchase set per customer, the product ids bought in the last year, is kept in memory in the recommendation service and rebuilt from the purchases table nightly, with today's orders appended as they happen. The filter drops a candidate in the set unless the product's **repurchasable** flag is on, and for those it drops the candidate only if the last purchase is more recent than the product's typical repurchase interval, a number the nightly rebuild computes from the purchases table. A washing machine is dropped for a year; coffee comes back after three weeks.

The index is a dataset

The vector database carries a version, the tower version that embedded it, the catalog as-of time, its count, and its recall against an exact scan on a sample of customer vectors. A new product tower is a full re-index: every vector moves, the centroids are chosen again, and the pointer swaps the index and the customer tower together.

3. Measurement

Before any customer sees it

STAGE	MEASURE	AGAINST
the vector database	recall of the exact nearest 300 at the cells probed	an exact scan, on a sample of customer vectors, after every build
each candidate source	recall at 2,000: of the products the customer went on to buy, the share that was among its candidates	the impression log joined with purchases, with the exploration slot
the filters and the ordering	grids that violated a rule; a bought, non-repurchasable product shown	zero, or the rule is broken
the ranker	NDCG at 12, the first three rows, per slice	logged grids, with the shuffled slice for the position correction

Per slice: customers with no purchase history; the home page against the product page; each region; products under thirty days old.

With customers

An A/B test, customers assigned at random, for two weeks because purchases take a session and the signal is small. The **primary**: revenue per session from the grid. The **guardrails**, each with its own floor: the return rate four weeks later, the share of sessions that reach support, p99 latency of the grid, the share of grids with fifty products, and the share of purchases that came from products added this month, so that a ranker that narrows the catalog to what already sells is caught. Interleaving for the quick decisions between candidate rankers; the A/B test for the release.

A release moves when the primary cleared its floor and nothing in the guardrail table moved against it.

The pipeline's own measures

Index freshness (the age of the newest product the vector database can return), the version match between the towers and the index, the stock table's age, the trending list's age, p99 per stage, the share of grids with fifty products, impression coverage.

Bonus: trending in this location

A job every ten minutes over the views and purchases of the last hour, by region: each product's rate now against its rate over the last four weeks at this hour, and the top few hundred by lift become the region's trending list, a candidate source with its own reason attached. The weather feed is one more input: a rule, owned by merchandising, that lifts a category when the region's forecast crosses a threshold (snow:

shovels, salt, boots), before the purchases that would make it trend have happened. Measured like any source: its recall, and the share of purchases its candidates earn, per region, against the same source without the weather rule.

The one question: pinned promotions

The first row earns clicks because it is the first row. A pinned promotion in it earns the clicks of its position and writes impression rows that look like the ranker's successes. Trained on naively, the ranker learns that promoted products are what customers want, promotes them itself, and next month's grid is the store's own merchandising preferences reflected back, with the slow truth, returns, arriving too late to stop it.

The design handles it in three places. The impression row carries the **source**, and rows whose source is "sponsored" or "pinned" are either excluded from the training set or carried with the source as a feature, so the ranker can learn that a pinned click is a different event. The **shuffled slice** measures the first row's own effect, so the correction does not credit the position to the product. And the **guardrail** on purchases from new products catches the narrowing if the first two fail.